

Automated Python-Based Image Analysis Pipeline for Segmentation and Quantification of Biomolecular Condensates in Fluorescence Microscopy Z-Stacks

Daniel Chang

C&S BIO 199 / 197 — Spring 2026

PI: Elisa Franco

1. Abstract

Biomolecular condensates are membrane-less cellular compartments formed through liquid-liquid phase separation that play critical roles in gene regulation and RNA processing. Recent work from the Franco lab has demonstrated that programmable RNA nanostar constructs can drive the formation of synthetic condensates in both the nuclear and cytoplasmic compartments of living mammalian cells. Quantifying the degree of condensate enrichment — expressed as the partition coefficient, the ratio of condensate-phase to dilute-phase fluorescence intensity — is essential for characterizing how RNA design parameters influence phase separation. However, existing measurement workflows rely on manual segmentation in GUI-based tools such as Imaris, which are difficult to scale across large datasets and introduce reproducibility concerns. This project develops a fully automated Python-based image analysis pipeline that addresses these limitations through Cellpose 3 denoising, full three-dimensional segmentation, and background-subtracted intensity estimation following the Fabrini et al. method. Applied to the JABr Sample2_5_1 region of interest, the pipeline produced a nuclear partition coefficient of 6.297 against a manually-derived reference of 6.324, a -0.4% error, demonstrating that automated analysis can quantitatively match expert manual measurement.

2. Introduction

Biomolecular condensates are membrane-less cellular compartments that play important roles in processes such as gene expression and RNA regulation. Recent work from the Franco lab has shown that artificial RNA condensates can be engineered in living cells using programmable RNA nanostar motifs, which form distinct compartments in both the nucleus and cytoplasm through sequence-specific interactions [1].

Fluorescence microscopy enables visualization of these condensates; however, extracting quantitative and reproducible measurements from multi-channel Z-stack imaging data remains challenging. Manual and GUI-based approaches (e.g., Imaris) are difficult to scale and lack reproducibility, particularly when analyzing large datasets or comparing results

across experimental conditions.

To address these limitations, this project develops an automated Python-based pipeline for condensate segmentation and partition coefficient quantification. The pipeline uses Cellpose 3 for denoising and 3D volumetric segmentation, followed by background-subtracted intensity estimation using the Fabrini et al. method. The goal is to produce partition coefficients that match the manually-derived reference values from the Franco lab dataset, enabling the pipeline to serve as a reproducible substitute for manual analysis.

3. Methods

3.1 Dataset Source and Biological Context

The fluorescence microscopy dataset used in this project was generated by Li et al. as part of the study "Programmable artificial RNA condensates in mammalian cells" [1], a preprint from the Franco lab at UCLA. The imaging data and corresponding manual partition coefficient measurements were shared directly with the author by Shiyi Li (first author) via Box file sharing. This dataset is not publicly available but was provided for use in this undergraduate research project under PI Elisa Franco.

The dataset comprises multi-channel confocal Z-stack images of HEK293T cells transfected with plasmids encoding RNA nanostar constructs. In the Li et al. study, partition coefficients for each construct were computed manually in Imaris for 30 cells, yielding per-cell condensate density, dilute density, and partition coefficient values split by cellular compartment (nuclear and cytoplasmic). These measurements serve as the reference validation target for the automated pipeline.

3.2 Construct Selection

The JABr construct was selected for this project on the recommendation of Shiyi Li, as it represents the reference nanostar design in Li et al. [1] and constitutes the most complete and well-characterized dataset. JABr is a three-arm RNA nanostar with 15-nucleotide (nt) arm length, kissing-loop variant A (UCGCGA), and the Broccoli fluorescent aptamer. The 15-nt arm length places the nanostar near the nuclear pore permeability threshold (approximately 5.1 nm; 66.3 kDa), causing it to form condensates in both the nucleus and the cytoplasm — a property that makes it particularly informative for studying compartment-specific enrichment [1].

3.3 Sample and Region of Interest Selection

A single field of view from the JABr dataset — referred to here as JABr-Sample2_5_1 — was selected for pipeline development and validation. This sample is a 55-slice multi-channel confocal Z-stack imaging a single HEK293T cell expressing the JABr

construct and stained with DFHBI to visualize the Broccoli-labeled RNA condensates. The two channels correspond to nuclei signal (Channel 1, Hoechst) and condensate signal (Channel 2, DFHBI-Broccoli).

The decision to focus on a single region of interest was deliberate: by targeting one well-characterized measurement, the pipeline could be developed and tuned against a known reference value without introducing variability across multiple cells. The reference nuclear partition coefficient for this region is 6.324, providing a precise quantitative benchmark. A single contiguous sub-region of the field of view was manually cropped to contain the primary nucleus and its associated condensates, reducing computational cost and eliminating edge effects from surrounding cells.

This single-sample approach is acknowledged as a limitation of the current work. Cross-validation across additional JABr regions and other construct datasets (e.g., GABr, AABr) is an intended direction for future work.

3.4 Pipeline Overview

The automated pipeline follows a five-step workflow:

**Channel Separation → Denoising → Segmentation → Partition Coefficient
Computation → Output**

All steps are implemented in Python and executed through a single script with command-line arguments controlling key parameters. GPU acceleration was used throughout via CUDA on an NVIDIA RTX 4080.

3.5 Channel Separation

The two-channel Z-stack was split into independent single-channel stacks prior to analysis: one for the nuclei signal and one for the condensate signal. Each channel is a 3D volume of 55 Z-slices at 185×259 pixels with uint16 bit depth, loaded using the `tifffile` library [5].

3.6 Denoising

Both channels were denoised using the Cellpose 3 `DenoiseModel` [3, 4] prior to segmentation. This model is trained to suppress shot noise and background haze in fluorescence microscopy images while preserving object boundaries. Applying denoising before segmentation improves the detection of faint condensate structures and reduces spurious detections from noise peaks.

3.7 Segmentation

Instance segmentation was performed using Cellpose 3 [3, 4] with `do_3D=True`, which processes the full volumetric Z-stack jointly rather than operating slice-by-slice. This

produces spatially coherent 3D instance labels, ensuring each object is represented as a single contiguous region across all contributing Z-slices.

Condensate segmentation used the pretrained `cyto3` model with a cell probability threshold of 0.0 and GPU acceleration enabled.

Nuclei segmentation used the same `cyto3` model, followed by connected-component relabeling [6] to enforce a single label per nucleus. The raw Cellpose output produced 76 candidate labels; after cleanup, 5 distinct nuclei were retained in the ROI.

3.8 Partition Coefficient Computation

The nuclear partition coefficient was computed following the method described in Fabrini et al. [1], adapted for 3D volumetric masks:

Background subtraction:

$$B = \min(\text{all voxel intensities in the condensate stack})$$

This estimates the camera offset and removes it from all intensity measurements before computing densities.

Condensate density:

The condensate phase was defined as all voxels falling inside both the condensate mask and the nucleus mask — i.e., nuclear condensates. Mean background-subtracted intensity was computed over this intersection.

$$\rho_{\text{cond}} = \frac{\text{mean}(\text{clip}(I - B, 0))}{\text{cond mask} \cap \text{nuc mask}}$$

Dilute density:

The dilute phase was defined as voxels inside the nucleus mask but outside the condensate mask. To approximate the manual approach of selecting a quiet representative region, the 50 lowest-intensity valid $10 \times 10 \times 10$ voxel patches fully within the dilute region were identified, and their mean intensity was used as the dilute density estimate.

$$\rho_{\text{dilute}} = \frac{\text{mean of 50 lowest-intensity } 10 \times 10 \times 10 \text{ patches within}}{\text{nuc mask} \cap \neg \text{cond mask}}$$

Partition coefficient:

$$\text{PC} = \frac{\rho_{\text{cond}}}{\rho_{\text{dilute}}}$$

4. Results

4.1 Segmentation

The pipeline segmented 186 condensate objects and 5 nuclei in the Sample2_5_1 ROI. Nuclei were detected as spatially coherent 3D regions after connected-component relabeling reduced 76 raw Cellpose labels to 5 distinct nuclei. Condensates were identified as punctate 3D structures distributed throughout the nuclear volume.

4.2 Partition Coefficient

The pipeline produced the following measurements for Sample2_5_1:

Metric	Pipeline Output	Nuclear Reference	Error
Partition Coefficient	6.297	6.324	−0.4%
Condensate density	440.93	658.30	−33%
Dilute density	70.03	104.09	−33%

The partition coefficient of 6.297 is within 0.4% of the manual reference value of 6.324. Both the condensate density and dilute density are approximately 33% lower than the reference values, but because the underestimation is proportionally equal in both the numerator and denominator, the ratio — and thus the partition coefficient — is accurately recovered.

5. Discussion

The pipeline achieved a partition coefficient of 6.297, within 0.4% of the manual reference of 6.324. Two design choices were central to this accuracy.

Background subtraction (B = minimum voxel intensity across the stack) removes the camera offset from all intensity measurements. Without this correction, both condensate and dilute densities are overestimated, but not by the same factor — the dilute phase, which has lower absolute signal, is affected proportionally more, causing the PC to be underestimated.

Full 3D segmentation using `do_3D=True` produces spatially coherent instance labels across the entire Z-stack. This ensures each condensate object is represented as a single contiguous 3D region, so all contributing voxels are included in the intensity estimate.

The persistent ~33% offset in individual densities relative to the Imaris reference likely reflects differences in how the pipeline and Imaris define object boundaries. Despite this offset, the proportionality between condensate and dilute densities is preserved, yielding an accurate partition coefficient. Future work could investigate whether boundary definition differences can be corrected to also match individual density values.

A key limitation of this work is that the pipeline was validated on a single region of interest. While this was intentional — allowing focused development against a known reference — it

leaves open the question of whether the pipeline generalizes across other cells in the JABr dataset or to other nanostar constructs. Cross-validation across additional JABr regions and constructs such as GABr and AABr is a natural next step.

6. Conclusion

This project developed and validated an updated Python-based pipeline for automated quantification of biomolecular condensate partition coefficients from fluorescence microscopy Z-stacks. The key contributions of the spring quarter work are:

1. Integration of Cellpose 3 DenoiseModel for preprocessing
2. Full 3D volumetric segmentation (do_3D=True) replacing slice-by-slice analysis
3. Background subtraction following the Fabrini et al. method
4. Top-100% voxel density estimation for the condensate phase
5. Connected-component relabeling for nucleus cleanup

Applied to JABr Sample2_5_1, the pipeline achieved a partition coefficient of 6.297 against a manual reference of 6.324 (−0.4% error). These results establish the pipeline as a reproducible and accurate alternative to manual Imaris-based analysis for this sample.

7. Reproducibility

The pipeline is implemented entirely in Python using open-source libraries: Cellpose 3 for denoising and segmentation, NumPy and pandas for data processing, scikit-image for feature extraction, and tifffile for image I/O. All steps are executed through a single script with documented command-line arguments.

The pipeline was developed and tested on:

- **MacBook Pro** (Apple M1 Pro, 10-core CPU, 16 GB RAM)
- **Desktop workstation** (AMD Ryzen 7 7700 CPU, NVIDIA RTX 4080 GPU, 32 GB RAM)

GPU acceleration (CUDA, RTX 4080) was used for all segmentation and denoising steps reported here.

The full pipeline code is available at: https://github.com/Dauniel/research_lab

8. References

[1] Li, S., Kim, Y., Wang, K., Payson, E. J., Tang, A. A., Villalba Nieto, M., Osmanovic, D., Yang, M., Dilao, D., Bermudez, A., Xiao, W., Li, M. M. H., Lin, N. Y. C., Plath, K., Black, D.

L., & Franco, E. (2026). Programmable artificial RNA condensates in mammalian cells. **bioRxiv**. <https://doi.org/10.64898/2026.01.28.702393>

[2] Banani, S. F., Lee, H. O., Hyman, A. A., & Rosen, M. K. (2017). Biomolecular condensates: organizers of cellular biochemistry. **Nature Reviews Molecular Cell Biology**, 18(5), 285–298. <https://doi.org/10.1038/nrm.2017.7>

[3] Stringer, C., Wang, T., Michaelos, M., & Pachitariu, M. (2021). Cellpose: a generalist algorithm for cellular segmentation. **Nature Methods**, 18(1), 100–106. <https://doi.org/10.1038/s41592-020-01018-x>

[4] Pachitariu, M., & Stringer, C. (2022). Cellpose 2.0: how to train your own model. **Nature Methods**, 19(12), 1634–1641. <https://doi.org/10.1038/s41592-022-01663-4>

[5] Gohlke, C., et al. (2023). tiff file: Read and write TIFF files. Zenodo. <https://doi.org/10.5281/zenodo.6795860>

[6] van der Walt, S., et al. (2014). scikit-image: Image processing in Python. **PeerJ**, 2, e453. <https://doi.org/10.7717/peerj.453>